ROUND 20 — CONSTITUTION V7: THE SELF-REVISING INSTITUTIONAL MODEL

The Self-Revising Institutional Model

V6 adapts behavior. V7 adapts THEORY. The closed learning loop: reality disagrees with Maestro, model updates, better recommendation, better outcome.

Reviewer	Independent (Super Z, Z.ai)
Baseline	Commit a044a4a. V5 Phase 1 complete. V5 Specs #4-#8 NOT built. V6 NOT started. 63 backend modules, 27 frontend files.
V7 shift	V6 adapts the organization's behavior. V7 adapts Maestro's own THEORY of how the organization works. The model revises itself when reality disagrees. Maestro stops being an adaptive system and becomes a compounding institutional asset.
Three constitutional laws	Law 1: "Every interaction must permanently improve the organization." Law 2: "The organization becomes more intelligent even when nobody opens Maestro." Law 3 (NEW): "Every recommendation must make Maestro's understanding of the organization more accurate."
Deliverable	6 specifications across 3 phases: (1) Institution Model + governance modes, (2) closed learning loop + model revision, (3) living organizational model + compounding judgment. Each has API + acceptance test + build order.

WHY V7 EXISTS

V6 introduced adaptation: Maestro quietly restructures work, tracks eliminated failure modes, and produces the organization's autobiography. But V6 adapts BEHAVIOR — it changes what the organization does. It does not adapt its own THEORY of how the organization works. After two years, V6 Maestro knows "Engineering trusts Legal." But it does not know "Engineering trusts Legal ONLY when security reviews are finished before sprint planning, because three previous coordination failures established that norm." That is an internal organizational theory — a causal model that explains WHY, not just WHAT.

V7 is the self-revising layer. V7 says: every time reality disagrees with Maestro's prediction, the model updates. Every recommendation makes Maestro's understanding more accurate. The learning loop closes: signal → recommendation → outcome → model updated → better recommendation → better outcome. Maestro stops being an adaptive system and becomes a compounding institutional asset — one that gets smarter about THIS specific organization every day, in a way that cannot be copied because it is built from years of resolved disagreements between Maestro and reality.

The progression is now complete: V3 Observe → V4 Judge → V5 Disappear → V6 Adapt → V7 Understand → V8 Evolve. V7 is "Understand" — the institution model that continuously revises its causal theory of how the organization works. V8 will be "Evolve" — the point where the accumulated judgment becomes irreplaceable and the product stops being primarily software and starts being an institutional asset.

Three constitutional laws (all mandatory):

Law 1 (from V6): "Every interaction must permanently improve the organization."

Law 2 (from V6): "The organization should become more intelligent even when nobody opens Maestro."

Law 3 (NEW in V7): "Every recommendation must make Maestro's understanding of the organization more accurate." This closes the learning loop. After each recommendation resolves (hit or miss), the model revises. Reality disagrees → model updates → next recommendation is better. This is genuine organizational learning — not calibration (adjusting confidence), but theory revision (adjusting understanding).

The strategic risk principle (NEW in V7): Enterprises are extremely sensitive to autonomous changes. V7 introduces three governance modes: Recommendation ("I suggest..."), Execution ("I've prepared the workflow"), and Autonomous Execution ("I've already changed it"). Each mode has explicit approval boundaries. This makes the system adoptable in regulated enterprises while preserving the long-term vision.

The 6 V7 Specifications

V7 has 6 specs in 3 phases. Phase 0 is the V5+V6 backlog (must complete first). Phase 1 creates the Institution Model and governance modes. Phase 2 closes the learning loop. Phase 3 creates the living organizational model and compounding judgment. Total ~18 days after V5+V6 backlog.

#	Specification	Phase	Effort	What It Does
0	V5 Backlog (#4-#8) + V6 Specs (#1-#6)	0	~24 days	Forgetting, Causal, Temporal, Imagination, Recall + Adaptive Nudges, Evolution Tracker, Backgr
1	Institution Model	1	3 days	A continuously revised causal model answering "How does THIS organization actually work?"
2	Three-Mode Governance	1	2 days	Recommendation / Execution / Autonomous Execution with explicit approval boundaries
3	Closed Learning Loop	2	2.5 days	Reality disagrees with Maestro -> model updates -> better recommendation
4	Model Revision Engine	2	2 days	When a prediction is resolved, revise the institution model's causal theory
5	Living Organizational Model	3	3 days	Merged DNA + Narrative + Institution Model into one self-revising substrate
6	Compounding Judgment Tracker	3	1.5 days	Tracks whether Maestro's recommendations are getting MORE accurate over time
TOTAL	**6 V7 specs (after V5+V6 backlog)**	**3 phases**	**~14 days**	**V7: from adaptive system to compounding institutional asset**

Build order: Phase 0 (V5+V6 backlog, ~24 days, MANDATORY) -> Phase 1 (Institution Model + Governance, ~5 days) -> Phase 2 (Closed Loop + Model Revision, ~4.5 days) -> Phase 3 (Living Model + Compounding Judgment, ~4.5 days). Total ~14 days after backlog. Do NOT start V7 until V5+V6 backlog is complete.

Phase 0 — V5+V6 Backlog (MANDATORY Prerequisite)

V7 specs build on V5 (causal cognition, temporal trajectories, institutional recall) and V6 (adaptive nudges, evolution tracker, organizational DNA, autobiography). NONE of these exist yet. The Institution Model (V7 Spec #1) requires causal chains (V5 #6) and temporal trajectories (V5 #7). The Closed Learning Loop (V7 Spec #3) requires the evolution tracker (V6 #2). The Living Organizational Model (V7 Spec #5) merges DNA (V6 #5) and the autobiography (V6 #6).

PHASE 0 IS MANDATORY. DO NOT BUILD V7 ON A MISSING FOUNDATION.

V5 Specs #4-#8 are NOT BUILT (verified at commit a044a4a). V6 Specs #1-#6 are NOT BUILT. V7 specs build on ALL of them. The Institution Model needs causal chains. The Closed Learning Loop needs the evolution tracker. The Living Model needs DNA + autobiography. Build V5 Phase 2-3 first (see V5 Phase 2-3 Coding Instructions). Then build V6 Phase 1-3 (see V6 spec). Then build V7. Skipping ahead will produce a model that looks like it works but has no causal foundation.

Estimated backlog: ~24 days (9 days V5 Phase 2-3 + 15 days V6 Phase 1-3). V7 itself is ~14 days. Total to V7 completion: ~38 days from current state.

Phase 1 — Institution Model + Governance Modes

Phase 1 is the V7 foundation. The Institution Model is not another engine — it is the continuously revised causal model that answers "How does THIS organization actually work?" It sits underneath DNA, nudges, trajectories, and narratives. Governance modes make autonomous action safe for regulated enterprises.

SPEC #1 Institution Model — "How does THIS organization actually work?"

V7 Principle

V7: "What's missing is an explicit internal model. A continuously revised causal model answering: How does THIS organization actually work? Not how do organizations work — THIS one." After two years, Maestro should know: "Engineering trusts Legal ONLY when security reviews are finished before sprint planning, because three previous coordination failures established that norm." That is an internal organizational theory.

Current codebase gap

model.py (ExecutionModel) is a static accumulator — signals are added, counts increase, but the model does not revise its causal understanding. personality.py infers traits but does not explain WHY. wisdom.py synthesizes judgment but does not maintain a causal theory. The gap: no module maintains a causal model of how the specific organization works — its norms, causal pathways, incentives, trust dynamics, informal power, bottlenecks, and hidden dependencies. Maestro knows WHAT happens but not WHY it happens in THIS organization.

Files to create/modify

CREATE backend/maestro_oem/institution_model.py — the InstitutionModel. A continuously revised causal model with 5 layers: (1) Norms ("Engineering waits for Legal before deploying auth changes"), (2) Causal Pathways ("Legal review before launch causes 27% fewer post-launch bugs"), (3) Incentives ("Engineers prioritize velocity because sprint velocity is the primary review metric"), (4) Trust Dynamics ("Engineering trusts Legal only when security reviews precede sprint planning"), (5) Hidden Dependencies ("The OAuth migration depends on Alice's institutional knowledge of the legacy auth flow"). Each layer is populated from: causal.py (causal chains), evidence_graph.py (dependencies), contradiction.py (norms from contradictions), personality.py + identity.py (incentives from drift). The model is REVISED on every prediction resolution (see Spec #4). CREATE GET /api/oem/institution-model endpoint. MODIFY static/js/cognition.js — add a "How your organization works" section (calm narrative, not a dashboard) that renders the institution model as human sentences.

API contract

GET /api/oem/institution-model returns: { "norms": [{"norm": "Engineering waits for Legal before deploying auth changes", "established_by": "3 coordination failures in Q3 2024", "evidence_count": 5, "confidence": 0.82}], "causal_pathways": [{"pathway": "Legal review before launch -> 27% fewer post-launch bugs", "sequence_count": 3, "confidence": 0.91, "source": "causal.py chain causal-xxx"}], "incentives": [{"incentive": "Engineers prioritize velocity because sprint velocity is the primary review metric", "evidence_count": 8, "inferred_from": "6 of 8 PRs merged without full review"}], "trust_dynamics": [{"relationship": "Engineering -> Legal", "condition": "trust is high ONLY when security reviews precede sprint planning", "evidence_count": 4}], "hidden_dependencies": [{"dependency": "OAuth migration depends on Alice's institutional knowledge of legacy auth flow", "risk": "Alice departure would block the migration", "evidence_count": 3}], "narrative": "Your organization works as follows: Engineering prioritizes velocity (sprint velocity is the primary metric). Legal review is a norm, not a rule — it is followed when security reviews precede sprint planning. The OAuth migration has a hidden dependency on Alice. Three coordination failures in Q3 established the Engineering-Legal review norm." }. Must have at least 1 item in each of the 5 layers with evidence_count > 0 (or honest "insufficient data for this layer").

Acceptance test (auditor will run)

1) GET /api/oem/institution-model returns 5 layers, each with at least 1 item (or honest "insufficient data"). 2) Each item has evidence_count > 0 and references real model data (not hardcoded). 3) The narrative synthesizes the layers into a "how your organization works" paragraph. 4) Cognition surface shows a "How your organization works" section with human sentences. 5) No organ names, no jargon. 6) V5 litmus: no new panel — enhances existing Cognition surface. 7) V7 litmus: does this make Maestro's understanding more accurate? YES — it is the model OF the organization.

Effort

3 days (2 days model construction from existing data + 0.5 day API + 0.5 day frontend)

Dependencies

V5 #6 (causal.py) for causal pathways. V5 #7 (temporal_trajectories.py) for norm establishment timing. V4 identity.py for incentives. V4 contradiction.py for norms. V6 #5 (organizational DNA) for trust dynamics baseline.

SPEC #2 Three-Mode Governance — Recommendation / Execution / Autonomous Execution

V7 Principle

V7: "Enterprises are extremely sensitive to autonomous changes. I would distinguish three modes: Recommendation (I suggest), Execution (I've prepared the workflow), Autonomous Execution (I've already changed it). Each has very different governance, risk, and compliance implications."

Current codebase gap

No governance modes exist. The executive_function.py engine produces a plan but does not distinguish between suggesting, preparing, and autonomously executing. The adaptive_nudge.py engine (V6, not yet built) will propose work restructuring but has no governance framework for approval levels. The gap: Maestro has no concept of approval boundaries — it either suggests or doesn't, with no middle ground of "prepared but not executed."

Files to create/modify

CREATE backend/maestro_oem/governance.py — the GovernanceManager. Defines 3 modes: (1) RECOMMENDATION — Maestro suggests an action; the user must approve and execute. (2) EXECUTION — Maestro prepares the artifact (drafted memo, calendar invite, workflow change document) but does not apply it; the user reviews and clicks "Apply." (3) AUTONOMOUS_EXECUTION — Maestro applies the change directly (e.g., updates the approval routing in Jira); the user is notified after the fact. Each mode has: a confidence threshold (RECOMMENDATION: any, EXECUTION: confidence >= 0.7, AUTONOMOUS: confidence >= 0.9 AND causal sequence_count >= 5), an approval requirement (RECOMMENDATION: none, EXECUTION: user clicks "Apply," AUTONOMOUS: no approval, but logged + reversible), and a reversibility flag (all modes are reversible; autonomous changes are logged in an immutable audit trail). CREATE GET /api/oem/governance/modes endpoint (shows available modes + thresholds). MODIFY backend/maestro_oem/executive_function.py — each execution plan includes a governance_mode field. MODIFY backend/maestro_oem/adaptive_nudge.py (V6, to be built) — each nudge specifies its governance mode. MODIFY static/js/today.js — the "Prepare" button now shows the governance mode: "Maestro suggests" (Recommendation), "Maestro has prepared" (Execution), "Maestro has updated" (Autonomous). Autonomous changes include a "Revert" button.

API contract

GET /api/oem/governance/modes returns: { "modes": [{"mode": "recommendation", "description": "Maestro suggests. You decide and execute.", "confidence_threshold": 0.0, "approval_required": false, "reversible": true}, {"mode": "execution", "description": "Maestro prepares. You review and apply.", "confidence_threshold": 0.7, "approval_required": true, "reversible": true}, {"mode": "autonomous", "description": "Maestro applies. You are notified.", "confidence_threshold": 0.9, "approval_required": false, "reversible": true, "audit_logged": true}], "current_defaults": {"recommendation": true, "execution": true, "autonomous": false}, "summary": "Autonomous execution is disabled by default. Enable it in Settings when the organization is ready for Maestro to make changes on its own." }.

Acceptance test (auditor will run)

1) GET /api/oem/governance/modes returns 3 modes with thresholds + approval requirements. 2) executive_function.py plans include governance_mode field. 3) Autonomous mode is DISABLED by default (safety). 4) Autonomous changes are logged in an audit trail. 5) All changes are reversible (Revert button). 6) TODAY shows governance mode labels: "Maestro suggests" / "Maestro has prepared" / "Maestro has updated." 7) V5 litmus: no new panel — labels on existing Prepare button. 8) V7 litmus: does this make Maestro's understanding more accurate? Indirectly — it makes Maestro's ACTIONS governed, which is a prerequisite for the learning loop (Spec #3).

Effort

2 days (1 day governance engine + 0.5 day API + 0.5 day frontend labels + audit trail)

Dependencies

V5 #2 (executive_function.py) for plan integration. V6 #1 (adaptive_nudge.py) for nudge integration.

Phase 2 — Closed Learning Loop + Model Revision

Phase 2 is the V7 core. The closed learning loop is what separates an adaptive system from a compounding institutional asset. When reality disagrees with Maestro's prediction, the model revises. The next recommendation is better. Over years, the accumulated revisions become irreplaceable organizational judgment.

SPEC #3 Closed Learning Loop — reality disagrees, model updates, better recommendation

V7 Principle

V7 Law 3: "Every recommendation must make Maestro's understanding of the organization more accurate." Current loop: Signal -> Recommendation -> Adaptation -> Improvement. V7 loop: Signal -> Recommendation -> Outcome -> Model updated -> Better recommendation -> Better outcome. Maestro improves itself because reality disagreed with it.

Current codebase gap

prediction_lifecycle.py (859 lines) resolves predictions as hit/miss. learning.py (993 lines) tracks calibration (Brier score, SHR). confidence.py (472 lines) adjusts confidence based on historical accuracy. BUT: none of these UPDATE THE MODEL'S CAUSAL THEORY after a prediction is resolved. Calibration adjusts confidence (how sure am I?) but not understanding (how does the org work?). The gap: when Maestro predicts "this bottleneck will cause a velocity drop" and it DOESN'T (miss), the confidence drops — but the causal model is not revised. Maestro does not learn "my theory of why bottlenecks cause velocity drops was wrong in this case because..."

Files to create/modify

CREATE backend/maestro_oem/closed_learning_loop.py — the ClosedLearningLoop. On every prediction resolution (hooked into prediction_lifecycle.py _resolve()), the loop: (1) compares the predicted outcome to the actual outcome, (2) if MISS (prediction was wrong), identifies which causal assumption was incorrect, (3) calls institution_model.py to revise the assumption, (4) logs the revision (what changed, why, what the old theory was, what the new theory is), (5) updates the confidence calculator to reflect the revised understanding. CREATE GET /api/oem/learning-loop endpoint (shows recent revisions). MODIFY backend/maestro_oem/prediction_lifecycle.py _resolve() — call the closed learning loop after each resolution. MODIFY static/js/cognition.js — add a "What Maestro learned" section showing recent model revisions: "Maestro predicted the auth bottleneck would cause a velocity drop. It didn't — because Alice ad-hoc reviewed the PRs outside the formal workflow. Maestro revised: 'Bottlenecks cause velocity drops ONLY when no informal reviewer exists. Alice's informal review is a hidden dependency.'"

API contract

GET /api/oem/learning-loop returns: { "revisions": [{"prediction_id": "pred-xxx", "predicted": "Auth bottleneck will cause 20% velocity drop", "actual": "No velocity drop — Alice reviewed PRs informally", "outcome": "miss", "revised_assumption": "Bottlenecks cause velocity drops ONLY when no informal reviewer exists", "old_theory": "Bottlenecks always cause velocity drops", "new_theory": "Bottlenecks cause velocity drops when no informal reviewer compensates", "revised_at": "...", "evidence_count": 1}], "total_revisions": 1, "accuracy_trend": [{"period": "2025-01", "accuracy": 0.72}, {"period": "2025-02", "accuracy": 0.78}], "narrative": "Maestro has revised its understanding 1 time. Its prediction accuracy improved from 72% to 78% after the revision. The organization is getting easier to predict." }. Must have at least 1 revision (or honest "no predictions have been resolved yet — the learning loop has not triggered").

Acceptance test (auditor will run)

1) GET /api/oem/learning-loop returns revisions array + accuracy_trend + narrative. 2) Each revision has predicted, actual, outcome, revised_assumption, old_theory, new_theory. 3) Auditor verifies the loop runs on prediction resolution (check prediction_lifecycle.py _resolve() calls the loop). 4) Auditor resolves a prediction (via POST /api/oem/predictions/resolve) and verifies a revision is created (if the prediction was a miss). 5) Cognition surface shows "What Maestro learned" section. 6) V7 litmus: does this make Maestro's understanding more accurate? YES — the model literally revises when it is wrong.

Effort

2.5 days (1.5 days revision logic + 0.5 day API + 0.5 day frontend)

Dependencies

V7 #1 (Institution Model) for the model that gets revised. V5 #6 (causal.py) for causal assumptions. prediction_lifecycle.py for resolution hook.

SPEC #4 Model Revision Engine — when a prediction resolves, revise the causal theory

V7 Principle

V7: "Maestro still adapts behavior. It doesn't yet adapt its own theory of the organization. Those are different things." The Model Revision Engine is the mechanism: when a prediction is resolved as a miss, it identifies which causal assumption was wrong and revises the institution model.

Current codebase gap

The closed learning loop (Spec #3) detects misses and calls for revision. But the REVISION LOGIC — identifying which assumption was wrong and how to update it — does not exist. The gap: Maestro knows it was wrong but not HOW to revise its theory. This spec creates the revision logic: for each miss, trace the prediction back to its causal assumptions (via causal.py), identify which assumption was violated by the actual outcome, and generate a revised assumption.

Files to create/modify

CREATE backend/maestro_oem/model_revision.py — the ModelRevisionEngine. For each prediction miss: (1) trace the prediction to its linked causal chains (from causal.py), (2) identify which causal assumption the prediction depended on, (3) compare the predicted causal pathway to the actual outcome, (4) generate a revised assumption that accounts for the miss, (5) update institution_model.py with the revised assumption, (6) log the revision (old theory, new theory, evidence). The revision logic uses the actual outcome to identify the missing variable: "Maestro predicted velocity would drop because of the bottleneck. It didn't. The missing variable was Alice's informal review. Revised: bottlenecks cause velocity drops ONLY when no informal reviewer compensates." CREATE GET /api/oem/model-revisions endpoint (shows revision history). MODIFY backend/maestro_oem/closed_learning_loop.py — call the revision engine on each miss.

API contract

GET /api/oem/model-revisions returns: { "revisions": [{"id": "rev-001", "triggered_by": "pred-xxx (miss)", "old_assumption": "Bottlenecks always cause velocity drops", "new_assumption": "Bottlenecks cause velocity drops when no informal reviewer compensates", "missing_variable_identified": "Alice's informal PR review", "evidence_for_revision": "Prediction pred-xxx missed because Alice reviewed 3 PRs outside the formal workflow", "revised_at": "...", "institution_model_layer": "causal_pathways"}], "total_revisions": 1, "revision_rate": "1 revision per 12 predictions", "narrative": "Maestro has revised its causal theory 1 time. The revision identified a hidden variable (Alice's informal review) that the original theory missed. Future predictions about bottlenecks will account for this variable." }.

Acceptance test (auditor will run)

1) GET /api/oem/model-revisions returns revisions with old_assumption, new_assumption, missing_variable_identified. 2) Each revision references the prediction that triggered it. 3) The new_assumption is MORE SPECIFIC than the old (it adds a condition). 4) Auditor triggers a miss (resolve a prediction as incorrect) and verifies a revision is created. 5) institution_model.py is updated (the causal_pathways layer reflects the new assumption). 6) V7 litmus: does this make Maestro's understanding more accurate? YES — each revision adds a condition that makes the theory more precise.

Effort

2 days (1.5 days revision logic + 0.5 day API)

Dependencies

V7 #1 (Institution Model) for the model being revised. V7 #3 (Closed Learning Loop) for the trigger. V5 #6 (causal.py) for causal assumptions to revise.

Phase 3 — Living Organizational Model + Compounding Judgment

Phase 3 merges everything. The Living Organizational Model unifies DNA + Narrative + Institution Model into one self-revising substrate. The Compounding Judgment Tracker proves that Maestro's recommendations are getting MORE accurate over time — the evidence that the product is a compounding institutional asset, not just software.

SPEC #5 Living Organizational Model — merged DNA + Narrative + Institution Model

V7 Principle

V7: "DNA explains who you are. Narrative explains how you became that. They're two projections of the same underlying model. I'd evolve the idea toward a Living Organizational Model." The Living Model unifies DNA (static identity), Narrative (how it evolved), and Institution Model (how it works) into one continuously revised substrate.

Current codebase gap

V6 Spec #5 (Organizational DNA, not yet built) would infer 7 chromosomes. V6 Spec #6 (Evolution Narrative, not yet built) would produce chapters. V7 Spec #1 (Institution Model) maintains causal theory. Currently these are 3 separate modules with no integration. The gap: DNA, Narrative, and Institution Model are three projections of the same underlying reality but are not unified. A user asking "who is my organization?" gets different answers from DNA ("consensus-driven, cautious") vs Institution Model ("Engineering waits for Legal") vs Narrative ("started fast, learned to seek consensus"). The Living Model unifies them.

Files to create/modify

CREATE backend/maestro_oem/living_model.py — the LivingOrganizationalModel. Composes: (1) DNA (from organizational_dna.py, V6 #5) — who the org IS, (2) Institution Model (from institution_model.py, V7 #1) — HOW the org WORKS, (3) Narrative (from evolution_narrative.py, V6 #6) — HOW the org BECAME what it is, (4) Revision history (from model_revision.py, V7 #4) — how Maestro's UNDERSTANDING has evolved. Produces a unified model: {identity (DNA), mechanics (Institution Model), history (Narrative), self_awareness (revision history)}. CREATE GET /api/oem/living-model endpoint. MODIFY static/js/learn.js — replace separate DNA, Identity, and Evolution sections with a single "Your organization" section that renders the living model as a coherent narrative (not separate panels).

API contract

GET /api/oem/living-model returns: { "identity": {"decision_style": "consensus", "risk_appetite": "cautious", "learning_velocity": "fast", "narrative": "Your organization decides by consensus, takes moderate risks, and learns quickly."}, "mechanics": {"norms": ["Engineering waits for Legal before auth deployments"], "causal_pathways": ["Legal review -> 27% fewer bugs"], "hidden_dependencies": ["OAuth depends on Alice"], "narrative": "Your organization works as follows: Engineering prioritizes velocity but follows the Legal review norm. The OAuth migration has a hidden dependency on Alice."}, "history": {"chapters": [{"title": "The Fast Months", "period": "2024-Q3", "narrative": "Your organization believed it was fast. It wasn't."}], "narrative": "Your organization started fast but fragile. It learned that review produces better outcomes."}, "self_awareness": {"total_revisions": 1, "accuracy_trend": "improving", "narrative": "Maestro has revised its understanding 1 time. Its predictions are getting more accurate."}, "unified_narrative": "Your organization is consensus-driven and cautious. It works by following norms (Engineering waits for Legal) that were established by past failures. It started fast but learned to seek review. Maestro's understanding of the organization has improved through 1 revision." }. Must have all 4 sections with non-empty narratives.

Acceptance test (auditor will run)

1) GET /api/oem/living-model returns identity, mechanics, history, self_awareness, unified_narrative. 2) Each section has a narrative (not a metric dump). 3) The unified_narrative synthesizes all 4 into a coherent paragraph. 4) LEARN surface shows a single "Your organization" section (replaces separate DNA + Identity + Evolution panels). 5) V5 litmus: SIMPLER (3 panels merged into 1). 6) V7 litmus: does this make Maestro's understanding more accurate? YES — it unifies all projections into one self-consistent model.

Effort

3 days (2 days composition engine + 0.5 day API + 0.5 day frontend merge)

Dependencies

V6 #5 (DNA) + V6 #6 (Narrative) + V7 #1 (Institution Model) + V7 #4 (Model Revision). ALL must be built first.

SPEC #6 Compounding Judgment Tracker — "Maestro's recommendations are getting more accurate"

V7 Principle

V7: "Once Maestro continuously updates its own causal understanding, you move from an adaptive system to one that compounds organizational judgment over years. That's the point where the product stops being primarily software and starts becoming an institutional asset." The Compounding Judgment Tracker proves this: it tracks whether Maestro's prediction accuracy is IMPROVING over time, and whether the model revisions are making it smarter.

Current codebase gap

learning.py (993 lines) tracks calibration (Brier score, hit rate). prediction_lifecycle.py tracks resolved predictions. But neither tracks whether accuracy is IMPROVING — they track current accuracy, not the trend. The gap: Maestro can say "my accuracy is 78%" but not "my accuracy has improved from 72% to 78% over 6 months, and 60% of that improvement is attributable to model revisions." The compounding judgment tracker proves the product is getting smarter — the evidence that it is an institutional asset, not just software.

Files to create/modify

CREATE backend/maestro_oem/compounding_judgment.py — the CompoundingJudgmentTracker. Tracks: (1) prediction accuracy over time (monthly buckets, from prediction_lifecycle.py), (2) accuracy improvement attributable to model revisions (compare accuracy before and after each revision, from model_revision.py), (3) the compounding rate (is the improvement accelerating, linear, or plateauing?), (4) the irreplacability score (how many years of resolved predictions + revisions would a competitor need to match Maestro's understanding?). CREATE GET /api/oem/compounding endpoint. MODIFY static/js/learn.js — add a "Maestro is getting smarter" section: "Maestro's prediction accuracy has improved from 72% to 78% over 6 months. 60% of the improvement is from model revisions. At the current rate, Maestro will be 85% accurate in 12 months. Replacing Maestro's understanding would require 2.3 years of organizational history."

API contract

GET /api/oem/compounding returns: { "accuracy_trend": [{"period": "2025-01", "accuracy": 0.72}, {"period": "2025-02", "accuracy": 0.74}, {"period": "2025-03", "accuracy": 0.78}], "improvement_rate": "+2% per month", "improvement_acceleration": "linear", "attributable_to_revisions": 0.60, "revisions_count": 3, "irreplaceability_score": {"years_of_history": 0.5, "resolved_predictions": 47, "model_revisions": 3, "estimated_replacement_time": "2.3 years", "narrative": "Replacing Maestro's understanding of your organization would require 2.3 years of organizational history, 47 resolved predictions, and 3 model revisions. This asset compounds over time."}, "narrative": "Maestro's prediction accuracy has improved from 72% to 78% over 6 months. 60% of the improvement is from model revisions. At the current rate, Maestro will be 85% accurate in 12 months." }. Must have accuracy_trend (3+ data points), irreplaceability_score with estimated_replacement_time.

Acceptance test (auditor will run)

1) GET /api/oem/compounding returns accuracy_trend, improvement_rate, irreplaceability_score, narrative. 2) accuracy_trend has 3+ data points showing improvement (or honest "insufficient history — need 3+ months of resolved predictions"). 3) irreplaceability_score includes estimated_replacement_time. 4) LEARN surface shows "Maestro is getting smarter" section. 5) V5 litmus: no new panel — enhances existing LEARN. 6) V7 litmus: does this make Maestro's understanding more accurate? It PROVES it does — the tracker is the evidence.

Effort

1.5 days (1 day tracking engine + 0.5 day API + frontend)

Dependencies

V7 #3 (Closed Learning Loop) + V7 #4 (Model Revision) for revision attribution. prediction_lifecycle.py for accuracy data.

Build Order and Dependencies

Phase	Specs	Duration	Why This Order	Unlocks
0	V5 #4-#8 + V6 #1-#6	~24 days	MANDATORY. V7 builds on causal chains, [illegible]	Causal + temporal + adaptation + DNA + evolution fa
1	#1 Institution Model + #2 Governance	~5 days	The model that gets revised. The governance that [illegible]	Self-revising model + safe to [illegible]
2	#3 Closed Learning Loop + #4 Model Revision	~4.5 days	The loop detects misses. The revision engine [illegible]	Closed loop: theory → [illegible]
3	#5 Living Model + #6 Compounding Judgment	~4.5 days	Unifies everything into one substrate. Proves the [illegible]	One model getting smarter + irreplaceability ev
TOTAL	**6 V7 specs (after backlog)**	**3 phases**	**~14 days after V5+V6**	**V7: from adaptive system to compounding in**

The Three Constitutional Laws (All Mandatory)

THREE LAWS, ALL IMMUTABLE

Law 1 (V6): "Every interaction must permanently improve the organization." Not the model. Not the UI. The organization. Checked by: does the spec create a mechanism for permanent change?

Law 2 (V6): "The organization should become more intelligent even when nobody opens Maestro." Checked by: does the spec run in the background, not on user request?

Law 3 (V7 NEW): "Every recommendation must make Maestro's understanding of the organization more accurate." Checked by: after the recommendation resolves, does the model revise? If the model does not revise when reality disagrees, the loop is open and Maestro is not learning — it is just calibrated. Calibration adjusts confidence. Revision adjusts understanding. V7 demands revision.

All three laws must pass for every V7 spec. The V5 litmus test ("is the UI simpler?") is retained. The V6 litmus test ("does this permanently improve the organization?") is retained. The V7 litmus test ("does this make Maestro's understanding more accurate?") is added. Four checks. No exceptions.

The Complete Progression: V3 to V8

Version	Verb	What Maestro Does	Moat
V3	Observe	Sees what happened. Reports patterns.	Data
V4	Judge	Synthesizes judgment. Recommends actions.	Cognitive organs
V5	Disappear	Becomes invisible. Acts through existing tools.	Invisible intelligence
V6	Adapt	Quietly restructures work. Prevents failures.	Institutional adaptation
V7	Understand	Revises its own theory when reality disagrees.	Self-revising institutional model
V8	**Evolve**	**Accumulated judgment becomes irreplaceable.**	**Compounding institutional asset**

V7 is "Understand." The moat shifts from adaptation ("we no longer make this mistake") to self-revising understanding ("we know WHY we no longer make this mistake, and our theory of the organization is more accurate than it was last month"). V8 is "Evolve" — the accumulated judgment becomes irreplaceable. V7 is the bridge: it makes Maestro's understanding compound over time, so that replacing Maestro means losing years of revised institutional theory.

Final Charge to the Coder

V6 made Maestro adapt the organization's behavior. V7 makes Maestro adapt its own THEORY of the organization. The closed learning loop — reality disagrees, model revises, better recommendation — is what separates an adaptive system from a compounding institutional asset. After V7, Maestro is not just software that helps the organization. It is an institutional model that gets more accurate every month, in a way that cannot be copied because it is built from years of resolved disagreements between Maestro and reality.

Build order: Phase 0 (V5+V6 backlog, ~24 days, MANDATORY) -> Phase 1 (Institution Model + Governance, ~5 days) -> Phase 2 (Closed Loop + Revision, ~4.5 days) -> Phase 3 (Living Model + Compounding Judgment, ~4.5 days). Total ~38 days from current state. Do NOT skip Phase 0. Do NOT build V7 on a missing foundation. The Institution Model needs causal chains. The Closed Loop needs the evolution tracker. The Living Model needs DNA + autobiography. Build the foundation first.

The V7 litmus test for every commit: "Does this make Maestro's understanding of the organization more accurate?" If yes, ship it. If it only adapts behavior without revising theory, it is V6 — useful but not V7. V7 specs must revise the model when reality disagrees. The bar is higher than V6: simpler UI AND permanent improvement AND background operation AND model revision. All four.

The end state is not software. It is an institutional asset that compounds judgment over years. Every resolved disagreement between Maestro and reality makes the model more accurate. Every revision is irreplaceable — a competitor cannot recreate it without living through the same organizational history. After V7, Maestro is not just an invisible intelligence layer or an adaptive system. It is the self-revising institutional theory of how THIS organization works — continuously updated, permanently improving, impossible to copy. That is what Apple, Microsoft, Google, or OpenAI would look at and think "we need to own this." Build it.